GPU-Accelerated Harris Corner Detection for Video Streams

Author Name(s)
Department / Institution
City, Country
email@domain.com

***Abstract*—This report presents a CUDA-based implementation of the Harris corner detector for video processing. The manuscript summarizes the algorithmic background, related literature, project architecture, and CUDA-oriented processing pipeline in IEEE format. It also outlines the planned experimental methodology for evaluating resolution-dependent performance in video workloads.**

***Index Terms*—Harris corner detection, CUDA, GPU acceleration, video processing, feature detection.**

I. Introduction

Corner detection is a core operation in computer vision pipelines including tracking, matching, motion analysis, and structure-from-motion. The Harris detector remains widely used due to its balance of computational cost, interpretability, and repeatability under moderate geometric and photometric variation. For high-frame-rate video, CPU implementations may become a bottleneck, motivating GPU acceleration.

This project implements a complete Harris pipeline with CUDA kernels for grayscale conversion, gradient estimation, second-moment matrix construction, Gaussian smoothing, response computation, and corner extraction. The implementation is integrated with OpenCV-based video input/output and frame-level visualization.

A practical and generic application scenario is *real-time corner-enhanced video preprocessing for downstream vision modules*. In this setting, Harris corners are overlaid or exported as lightweight geometric cues before optional tracking, motion analysis, or scene understanding stages.

Performance behavior across different resolutions (480p, 720p, and 1080p) will be investigated and reported in the later experimental sections of this paper.

II. Related Work (Literature Review)

A. Foundations of Corner Detection

Harris and Stephens introduced an auto-correlation-based criterion that identifies points with significant intensity changes in two orthogonal directions [1]. Their response measure, based on local gradient covariance, is computationally simple and has become a foundational building block for keypoint analysis.

Shi and Tomasi later proposed selecting features according to the minimum eigenvalue of the second-moment matrix [2], improving practical tracking stability for many scenes. In practice, Harris and Shi–Tomasi are often treated as neighboring methods on the same formulation family.

B. Scale and Invariance-Oriented Detectors

SIFT introduced robust scale-space keypoint detection and high-dimensional descriptors with strong matching performance under scale and rotation transformations [3]. SURF then targeted lower computational cost through approximations based on integral images [4].

These methods generally improve invariance and matching quality, but they can be heavier than Harris when the objective is low-latency detection in fixed or mildly varying imaging conditions.

C. Real-Time Feature Alternatives

ORB combines oriented FAST detection with rotated BRIEF descriptors and is widely used in real-time visual SLAM and embedded vision contexts [5]. Compared with Harris-based designs, ORB can provide strong runtime efficiency while trading off some sensitivity characteristics and descriptor assumptions.

D. GPU Vision Context

GPU execution is well-suited to this pipeline because its core stages (gradient computation, smoothing, and response evaluation) are data-parallel at the pixel level. Since each output depends on local neighborhoods with limited inter-pixel dependency, Harris is a natural candidate for CUDA optimization in streaming video workloads.

In summary, prior methods motivate different trade-offs between invariance, descriptor richness, and runtime. For this project, Harris is selected as a practical detector for frame-wise CUDA acceleration where low-latency corner extraction is prioritized over heavy descriptor computation.

E. Literature Comparison with This Project

Table I summarizes representative differences between foundational studies and the current project implementation.

The comparison in Table I highlights that the selected project direction emphasizes efficient detector-stage acceleration rather than descriptor-heavy matching. This positioning is aligned with the practical objective of frame-wise corner extraction on video streams, where deterministic execution

TABLE I
LITERATURE COMPARISON AND POSITIONING OF THE CURRENT CUDA HARRIS PROJECT

Study / Method	Primary focus	Typical trade-off	Relation to this project
Harris–Stephens [1]	Corner response from second-moment structure tensor.	Good efficiency and interpretability; limited scale invariance.	Core mathematical basis used in our CUDA frame pipeline.
Shi–Tomasi [2]	Tracking-oriented corner quality criterion (minimum eigenvalue).	Often stable for tracking; still classical local-feature assumptions.	Alternative corner scoring reference; could be used as an ablation baseline.
SIFT [3]	Scale-invariant keypoint detection + high-dimensional descriptors.	Strong robustness, but heavier computation and descriptor cost.	Included as a robustness-oriented reference; not selected for low-latency target.
SURF [4]	Faster approximation to SIFT using integral-image ideas.	Better speed than SIFT; still descriptor-heavy relative to Harris-only detection.	Useful comparison for speed/robustness balance, but not as lightweight as Harris response-only pipeline.
ORB [5]	Real-time oriented detector+binary descriptor.	Very fast matching pipeline; different detector/descriptor design goal.	Runtime-efficient alternative for future comparison; current project focuses on CUDA Harris response stages.
This project (CUDA Harris)	GPU-accelerated frame-wise Harris corner extraction for video.	Prioritizes detector throughput and modular kernel decomposition over descriptor invariance.	Targets resolution-dependent real-time behavior and straightforward integration into preprocessing workflows.

flow and kernel modularity are prioritized. Based on this context, the next section details how the repository and CUDA modules are organized to support this implementation strategy.

III. SYSTEM AND PROJECT STRUCTURE

A. Repository-Level Organization

The repository is structured around kernel-by-kernel decomposition, with interface headers in `include/` and CUDA implementation files in `src/`. The entry point coordinates video decoding, per-frame GPU processing, and output encoding. This separation improves maintainability and enables independent profiling of each computational stage.

B. Execution Pipeline

The video stream is first decoded into an ordered sequence of individual frames using host-side video I/O. Each decoded frame is treated as a standalone 2D image for GPU processing, while frame order is preserved to keep temporal consistency in the output video. After processing one frame, the corner-annotated result is returned to the output writer and the pipeline advances to the next frame.

Within each frame iteration, kernels are executed in a fixed dependency order and intermediate buffers are reused in device memory to reduce transfer overhead. Detailed per-kernel responsibilities are provided in the next subsection, while Figure 2 shows the stage-level flow.

C. Kernel Responsibilities

The following kernel-level descriptions define the functional responsibility of each CUDA module in the end-to-end Harris processing pipeline.

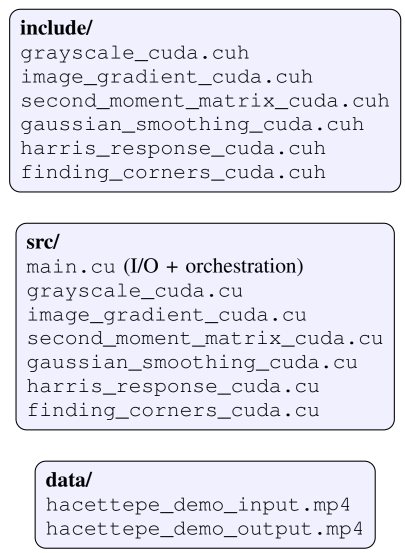

Fig. 1. Project/module structure used in the CUDA Harris pipeline implementation.

1) `grayscale_cuda.cu`: Converts input color frames to single-channel intensity values used by all downstream operations. This stage standardizes pixel representation, reduces memory bandwidth for subsequent kernels, and ensures derivative operators work on a consistent luminance domain rather than three independent color channels.

2) `image_gradient_cuda.cu`: Computes horizontal and vertical image derivatives (I_x and I_y), which form the basis of local structure analysis. In this project, Sobel filters are used for derivative estimation in both directions. The kernel maps one thread per pixel and applies local finite-difference style operations, producing gradient tensors that are consumed

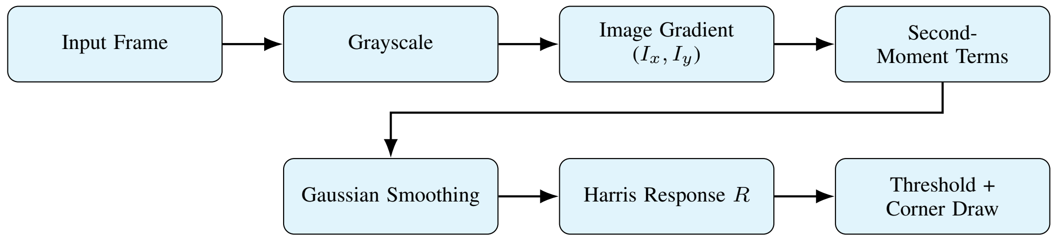

Fig. 2. Frame-wise CUDA processing flow from input frame to corner-annotated output (two-row stage layout).

directly by second-moment computation.

3) `second_moment_matrix_cuda.cu`: Builds per-pixel second-moment terms (I_x^2, I_y^2, and I_xI_y) before spatial smoothing. These quantities represent local directional energy and correlation, and they are stored in dedicated buffers to preserve precision through the subsequent smoothing stage.

4) `gaussian_smoothing_cuda.cu`: Applies Gaussian filtering to second-moment terms to stabilize neighborhood statistics and reduce noise sensitivity. By aggregating local evidence, this stage suppresses spurious gradient fluctuations and improves the numerical stability of the later Harris response evaluation.

5) `harris_response_cuda.cu`: Evaluates the Harris response function $R = \det(M) - k(\mathrm{trace}(M))^2$ for each pixel. The kernel combines smoothed tensor components into a scalar corner-strength map, where strong positive responses indicate candidate corner structures.

6) `finding_corners_cuda.cu`: Applies thresholding and corner-selection logic, then prepares output corner annotations. This stage filters weak responses, keeps salient locations, and writes overlay-ready coordinates/intensity cues for frame visualization in the final output stream.

IV. Method Details

A. Mathematical Formulation

Let an input RGB frame be denoted by channels $R(x, y)$, $G(x, y)$, and $B(x, y)$. The grayscale conversion used before derivative computation is

$$I(x, y) = 0.299\, R(x, y) + 0.587\, G(x, y) + 0.114\, B(x, y). \quad (1)$$

For filtering-oriented stages, padding is applied on the CPU side to create an extended image $\tilde{I}(x, y)$ so neighborhood reads remain valid at boundaries. Using this padded representation, derivative and smoothing operations follow standard local convolution forms:

$$I_x = K_x * \tilde{I}, \qquad I_y = K_y * \tilde{I}, \quad (2)$$

where K_x and K_y are Sobel derivative kernels (applied in horizontal and vertical directions, respectively) and $*$ denotes convolution. In this implementation, both Sobel and Gaussian filtering use 3×3 masks; these neighborhood operators require more computation than point-wise transforms, while still remaining lighter than larger filtering windows.

The second-moment matrix is then computed as

$$M = \begin{bmatrix} G_\sigma * I_x^2 & G_\sigma * I_xI_y \\ G_\sigma * I_xI_y & G_\sigma * I_y^2 \end{bmatrix}, \quad (3)$$

where G_σ is Gaussian smoothing. The Harris response is

$$R = \det(M) - k\,(\mathrm{trace}(M))^2. \quad (4)$$

Pixels with large positive R are interpreted as corners.

B. Implementation Notes

The staged decomposition in this project favors debuggability and selective optimization. The use of 3×3 Sobel and 3×3 Gaussian filters provides a practical balance between edge/corner quality and computational load. First, kernels can be validated independently against CPU-side references. Second, performance counters can be captured per stage to identify bottlenecks (e.g., memory bandwidth pressure in smoothing versus arithmetic intensity in response calculation). Third, parameter sweeps can be localized to one stage (e.g., Gaussian window size) without destabilizing the full pipeline. The implementation also applies CPU-side padding before filtering kernels to reduce boundary-branch complexity inside GPU convolutions.

Missing information to add:

- Numerical settings for k, Gaussian kernel size, and threshold.
- Border handling policy (replicate, reflect, clamp, etc.).
- Non-maximum suppression details (window size, tie-breaking).
- Exact device-memory layout and per-stage buffer reuse policy.
- Final numeric launch parameters used in experiments (block width/height and resulting grid dimensions).
- Measured stream-overlap benefit for non-filtering kernels (if reported in results).

V. Experimental Plan (Results Pending)

This section is intentionally designed to be completed after profiling and quality experiments.

A. Planned Metrics

- Runtime per frame (ms) and throughput (FPS) at multiple resolutions.
- End-to-end speedup versus a CPU baseline implementation.

- Temporal consistency of detected corner counts.
- Optional repeatability/localization quality on benchmark image pairs.

B. Planned Comparisons

- Harris GPU vs Harris CPU (same thresholds and smoothing settings).
- Harris GPU vs ORB/FAST detection runtime (if integration is available).
- Sensitivity to scene texture, motion blur, and illumination changes.

C. Planned Ablations

- Gaussian kernel size impact on noise robustness vs runtime.
- Harris threshold impact on precision-recall-like behavior.
- Resolution scaling behavior and memory footprint trends.

Missing information to add:

- GPU and CPU hardware/software configuration table.
- Dataset/video characteristics (resolution, duration, scene classes).
- Reproducible protocol (warm-up, run count, averaging, variance reporting).
- Whether timings are kernel-only or include host-device transfer and rendering.

VI. Threats to Validity and Limitations

Potential limitations include sensitivity to weak-texture regions, threshold dependence, and possible corner instability under severe blur or abrupt lighting changes. Experimental validity may also be affected by non-deterministic GPU scheduling noise, video codec variability, and inconsistent CPU frequency scaling.

Missing information to add:

- Any observed failure modes from your demo videos.
- Statistical confidence intervals for reported performance values.

VII. Conclusion and Next Steps

This draft now provides a more complete 6–8 page report foundation by expanding literature coverage, detailing project structure, and adding figures that clarify module layout and frame processing flow. The next revision should integrate measured results and comparative analysis to support final technical claims.

Missing information to add:

- Final quantitative conclusions from experiments.
- Optimization recommendations derived from profiling.
- Priority roadmap for further acceleration or robustness improvements.

References

[1] C. Harris and M. Stephens, "A combined corner and edge detector," in *Proceedings of the Alvey Vision Conference*, 1988, pp. 147–151.

[2] J. Shi and C. Tomasi, "Good features to track," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1994, pp. 593–600.

[3] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[4] H. Bay, A. Ess, T. Tuytelaars, and L. V. Gool, "Speeded-up robust features (surf)," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 346–359, 2008.

[5] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *Proceedings of IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2564–2571.